INFRASTRUCTURE AS CODE

Terraform

Professional Documentation

What it is, why teams use it, its advantages,
the HCL language, core concepts and best practices.

Declarative | Multi-cloud | HCL | DevOps

CREATED BY

Mohit Gadilohar

in www.linkedin.com/in/mohit-gadilohar-devops

12 November 2026

Terraform at a glance

Terraform is HashiCorp's Infrastructure as Code tool: you describe cloud and on-premises infrastructure in HCL configuration files, and Terraform creates, changes and destroys it to match.

2014
First release

HCL
Configuration language

Declarative
Describe the end state

Multi-cloud
Thousands of providers

Contents

1	What is Terraform?
2	Why use Terraform?
3	Advantages of Terraform
4	Which language does Terraform use?
5	How Terraform works
6	Core concepts
7	Common commands
8	Terraform compared with other tools
9	Limitations and licensing
10	Best practices
11	Sources

1 What is Terraform?

Terraform is an Infrastructure as Code (IaC) tool created by HashiCorp. It lets you define servers, networks, databases, DNS records, Kubernetes resources and SaaS settings as text files, then builds that infrastructure for you through the APIs of the platforms involved.

Infrastructure as Code means managing infrastructure through versioned, reviewable code instead of clicking through a cloud console or running ad-hoc scripts. The same files can be run again to rebuild an environment or to change it in a controlled way.

Terraform is **declarative**: you describe the end state you want (for example, three servers behind a load balancer), and Terraform works out which steps are needed to get there from the current state. It is **cloud-agnostic**, with providers for AWS, Microsoft Azure, Google Cloud, Oracle Cloud, Kubernetes, GitHub, Datadog and thousands of other services.

2 Why use Terraform?

Teams use Terraform because manual provisioning does not scale: it is slow, hard to repeat and easy to get wrong.

- **Repeatability:** the same code produces the same environment for development, testing and production.
- **Consistency:** every change goes through code, so environments stop drifting apart through one-off console edits.
- **Speed:** a full environment can be created or destroyed with one command instead of hours of manual steps.
- **Visibility before change:** `terraform plan` shows exactly what will be created, changed or destroyed before anything happens.
- **One workflow for many platforms:** the same language and commands work across AWS, Azure, Google Cloud and other services, which helps in multi-cloud and hybrid setups.
- **Collaboration and audit:** configuration lives in Git, so changes are reviewed in pull requests and every change has a history.
- **Cost control:** temporary environments can be torn down when no longer needed.

3 Advantages of Terraform

Advantage	What it means in practice
Declarative syntax	You state the desired result; Terraform calculates the steps and their order.
Multi-cloud support	One tool covers AWS, Azure, Google Cloud, Kubernetes and SaaS products through providers.
Execution plan	`terraform plan` previews every change, which reduces surprises in production.
Dependency graph	Terraform builds a graph of resource relationships, creates independent resources in parallel and orders dependent ones correctly.
State tracking	A state file records what Terraform manages, so it can detect drift and apply only the differences.
Reusable modules	Common patterns (a network, a cluster) are packaged once and reused across teams and projects.
Large ecosystem	The Terraform Registry hosts thousands of providers and community modules.
Version control friendly	Plain-text files work with Git, code review and CI/CD pipelines.
Idempotent	Applying the same configuration again makes no changes when infrastructure already matches.
Collaboration support	Remote state with locking lets teams work on the same infrastructure safely.

4 Which language does Terraform use?

Terraform configuration is written in **HCL (HashiCorp Configuration Language)**, saved in files with the `.tf` extension. HCL is a declarative language designed to be readable by humans while still being easy for tools to parse. Terraform also accepts a JSON variant (`.tf.json` files), which is mainly used when configuration is generated by other programs.

The Terraform core itself is written in Go. Providers, which talk to each platform's API, are also usually written in Go.

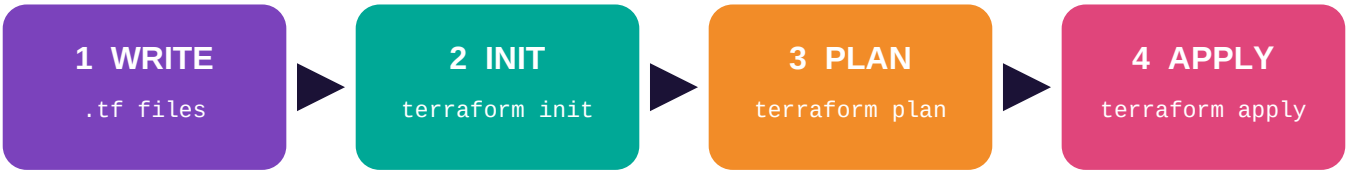
A minimal example that creates an AWS S3 bucket:

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.region
}

variable "region" {
  type    = string
  default = "ap-south-1"
}

resource "aws_s3_bucket" "logs" {
  bucket = "example-app-logs"

  tags = {
    Environment = "dev"
  }
}

output "bucket_arn" {
  value = aws_s3_bucket.logs.arn
}
```

The `terraform` block pins the provider, the `provider` block configures it, the `variable` makes the region changeable, the `resource` block declares the bucket, and the `output` block prints its ARN after apply.

5 How Terraform works

Terraform follows a four-step core workflow, and each step is a CLI command:



1. **Write:** define resources in `.tf` files.
2. **Init (`terraform init`):** download the required providers and modules and set up the backend that stores state.
3. **Plan (`terraform plan`):** compare your configuration with the recorded state and the real infrastructure, then show what would be created, updated or destroyed.
4. **Apply (`terraform apply`):** carry out the plan by calling the provider APIs, in dependency order, and update the state.

When the infrastructure is no longer needed, `terraform destroy` removes everything the configuration manages.

> **State file.** Terraform keeps a state file (`terraform.tfstate`) that maps each resource in your code to the real object in the cloud. State is how Terraform knows what already exists, so it can apply only the difference. For teams, state is normally stored in a remote backend (such as an S3 bucket, Azure Storage or Terraform Cloud) with locking, so two people cannot change the same infrastructure at once.

6 Core concepts

Concept	Purpose
Provider	A plugin that lets Terraform manage one platform, such as `aws`, `azurerm`, `google` or `kubernetes`.
Resource	A single infrastructure object you declare, such as a virtual machine, a bucket or a DNS record.
Data source	Read-only lookup of information that already exists, such as the latest AMI or an existing VPC.
Variable	An input that makes a configuration reusable across environments.
Output	A value exposed after apply, such as an IP address, or passed from one module to another.
Local value	A named expression used to avoid repeating logic inside a module.
Module	A folder of Terraform files packaged for reuse; every configuration is itself a root module.
State	The record of the real objects Terraform manages.
Backend	Where state is stored and how it is locked, such as local disk, S3 or Terraform Cloud.
Workspace	A separate state within the same configuration, often used for environments.
Provisioner	A last-resort way to run scripts on a resource after creation; prefer images or configuration tools instead.

7 Common commands

Command	What it does
`terraform init`	Prepares the working directory: downloads providers and modules and configures the backend.
`terraform fmt`	Rewrites files into the standard HCL style.
`terraform validate`	Checks the configuration for syntax and internal consistency errors.
`terraform plan`	Previews the changes needed to match the configuration.
`terraform apply`	Applies the changes after confirmation.
`terraform destroy`	Removes all resources managed by the configuration.
`terraform output`	Shows the output values from the state.
`terraform state list`	Lists the resources recorded in state.
`terraform import`	Brings an existing, manually created resource under Terraform management.
`terraform workspace`	Creates, lists and switches workspaces.

8 Terraform compared with other tools

Terraform is best at provisioning infrastructure across many platforms; it is not a replacement for configuration management or for every cloud-native tool.

Tool	Approach	Language	Typical use
Terraform	Declarative, multi-cloud provisioning with state	HCL	Creating and managing infrastructure on any platform with a provider
OpenTofu	Open-source fork of Terraform, same workflow	HCL	Teams that want a Linux Foundation-governed, open-source licence
AWS CloudFormation	Declarative, AWS only	JSON or YAML	Teams fully committed to AWS who want native integration
Pulumi	Declarative outcome defined with general-purpose code	Python, TypeScript, Go, C#, Java, YAML	Teams that prefer real programming languages and their tooling
Ansible	Mostly procedural configuration management, agentless	YAML playbooks	Installing software and configuring servers after they exist

Tip. Terraform and Ansible are often used together: Terraform creates the servers and networks, then Ansible configures what runs on them.

9 Limitations and licensing

Technical limitations

- **State is critical.** A lost or corrupted state file, or two people applying at once without locking, can cause real damage. Use a remote backend with locking.
- **Learning curve.** HCL is simple, but dependencies, state, modules and provider behaviour take time to learn.
- **Provider dependence.** Terraform can only manage what a provider supports; new cloud features may arrive in the provider after a delay.
- **Drift.** Changes made manually outside Terraform are only noticed at the next plan or refresh.
- **Not a configuration manager.** It provisions infrastructure but is not designed for managing software inside servers.
- **Secrets in state.** Sensitive values can be stored in plain text in state, so the state must be protected.

Licensing and ownership timeline

Date	Event
28 Jul 2014	First Terraform release (v0.1.0).
10 Aug 2023	HashiCorp announces a move to the Business Source License (BSL).
25 Aug 2023	A community fork, first called OpenTF and now OpenTofu, is announced.
20 Sep 2023	The Linux Foundation launches OpenTofu.

Date	Event
4 Oct 2023	Terraform 1.6.0, the first official release under the BSL.
10 Jan 2024	OpenTofu 1.6.0, its first stable release.
24 Apr 2024	IBM announces it will acquire HashiCorp.
27 Feb 2025	The IBM acquisition of HashiCorp is completed.

In practice, the BSL allows most internal and production use of Terraform, but restricts offering a competing commercial product built on it. Check the current licence text and ask your legal team if you build products or services on top of Terraform.

10 Best practices

- **Use remote state with locking** (for example S3 with a lock table, or Azure Storage) and never commit `terraform.tfstate` to Git.
- **Keep secrets out of code and state where possible.** Use a secrets manager, mark variables `sensitive`, and encrypt the state backend.
- **Pin versions** of Terraform and of every provider so that runs are reproducible.
- **Split configuration into modules** and keep one small state per environment or component to limit the blast radius of a mistake.
- **Run `fmt`, `validate` and `plan` in CI**, require review of the plan, and apply only from the pipeline.
- **Use consistent naming and tagging** so resources can be found and costs allocated.
- **Avoid manual changes** in the console; if one is unavoidable, reconcile it with `terraform import` or by updating the code.
- **Scan configuration** with tools such as tflint, Checkov or Trivy to catch misconfigurations before apply.
- **Always read the plan** before approving an apply, especially for resources marked for destruction.

11 Sources

- Terraform and OpenTofu history and timeline: dates for the release, licence change, OpenTofu fork and IBM acquisition (opened 12 November 2026).
- Concept, command and workflow descriptions are general product knowledge from the official Terraform documentation at developer.hashicorp.com/terraform; confirm details there before relying on them for production work.

Prepared by Mohit Gadilohar | www.linkedin.com/in/mohit-gadilohar-devops